

Exercice 1
Advanced Computer Architecture

Anton Schelfhout, Yari Vanherpe

12 februari 2024



Instructor: A. Marinšek

1 Introduction

In this lab session we switched the order of elements in an array. Conventionally this is done in a simple for loop using the CPU without the use of threads. This report describes an other approach using the GPU and different threads.

2 The project structure

In order to be able to use the GPU, we need to use CUDA. CUDA is a developed by NVIDIA and is used for parallel computing on the GPU.
The project is written in C++, and uses API calls from CUDA such as cudaMalloc (used to allocate memory in the RAM of the GPU).

In order to let the compiler know we want to execute a function on the CPU or GPU, we give it the "__global__"tag. For this lab we also could have used the __device__ tag, which means the function is only visible for the GPU.

When a function for the GPU is called, we use a special tag: "<<< amountOfBlocks, amountOfThreadsInBlock >>> ", this is done so the compiler knows how to structure the pool of threads. The threading pool is structured using blocks, containing multiple threads. The maximum amount of threads per block is 1024, and the amount of blocks is virtually unlimited.

3 Method

In order to be able to reverse the order of the array we need to have this array in the memory of the GPU. To do this we use the cudaMemcpy-function, to be able to copy to the GPU we also need memory to already be allocated there using cudaMalloc.

The GPU also writes the resulting array in memory which was also allocated beforehand, this array is than copied back in to system memory with the cudaMemcpy-function.

After the operation is done we don't need the allocated memory from the GPU so we free it using cudaFree.

4 Observations

Once we tried using an array bigger than 1024 but kept using 1 block the resulting array contained uninitialized data. This is because the GPU allocated memory for the array, but could not fill it because the block size was larger than the maximum. In order to fix this we just needed to use multiple blocks. We do this by dividing the size by 1024, and adding 1 to the result. This gives us the amount of blocks we need, the amount of threads in the block is 1024. We are aware there are more efficient ways to calculate the amount of threads per block, so there are less ïdle"threads in the last block (but more blocks overall).

Another interesting case is when for example we have a array with size 800, and we use 2 blocks of 500 threads. The last 200 threads of the last block will have an index larger than the size of the array, and write random data from behind the array to addresses in front of the array. This behaviour is not memory safe and should always be avoided, that is why we check if the index is larger than the size, and if so we don't switch the element.